

Politecnico di Torino

Department of Control and Computer Engineering (DAUIN)

Master of Science in Computer Engineering / Data Science

Large Language Models for Software Engineering
Official Examination Session – August 31, 2026**Course Code:** 01URQOV**Academic Year:** 2025/2026**Date:** Monday, August 31, 2026**Time Window:** 08:31 – 09:46 CEST**Duration:** 1 hour 15 minutes (75 minutes)**Total Questions:** 11 questions**Max Points Available:** 17.5 points (equivalent to 30L)**Exam Status:** Completed**Examination Instructions and Structure**

This official examination tests theoretical mastery, algorithmic implementation, prompt engineering, multi-agent systems design, and critical safety/fairness analysis of Large Language Models in Software Engineering.

- **Negative Marking:** Multiple-choice questions with explicit penalty impose a -25% penalty per incorrect response. Computational and open-ended design problems carry no negative penalty.
- **Structure:** The examination consists of two main parts:
 - **Part I: Exam Questions:** The unabridged text of all 11 examination questions (Domanda 1 through 11) with original point weightings, options, code snippets, and asset references.
 - **Part II: Comprehensive Solutions & In-Depth Analysis:** Publication-grade derivations, complete mathematical step-by-step calculations, production-grade agentic architectures, few-shot prompt designs, and critical bias evaluations.

1 Part I: Exam Questions**Question 1****[1.0 Point | Penalty: -0.25]**

Adapter layers are introduced to reduce the number of tunable parameters. They consist in:

- A method for splitting a large model into smaller sub-models that are trained independently and later combined
- A technique to merge multiple pre-trained models into a single model with fewer parameters
- Replacing the feedforward layers in Transformers with low-rank matrix decompositions to reduce memory usage
- Small, trainable layers inserted into a pre-trained model, which are fine-tuned instead of the rest of the original model
- Additional layers that replace the attention mechanism in the Transformer architecture

to simplify computation

- f. None of the others is a valid description of adapters
- g. A task-specific head added to the model, which is fine-tuned while the rest of the model is kept frozen

Question 2

[1.5 Points | No Penalty]

You are given a BERT-like, encoder-only model. You wish to fine-tune the model on a binary classification task, using a classification head taking as input the [CLS] vector. What is the total number of parameters that will be fine-tuned?

Note: Assume that none of the layers uses a bias term. You can ignore layer normalization parameters.

- Vocabulary size: $V = 128,000$
- Embedding/model dimension: $d = 32,768$
- Positional embeddings: absolute, learned, for 2048 positions
- Number of layers: $L = 24$
- Number of attention heads: $H = 64$
- Feed-Forward Network (FF-NN) projection dimension: $d_{ff} = 32,768$
- Key, query size: $d_k = 256$
- Value size: $d_v = 128$

Write the answer in the box below, rounded to the closest million.

Question 3

[1.5 Points | No Penalty]

You are given the following sequence of characters:

b e e e e a b e e a a a a

Initially, each character is a separate token. Apply Byte-Pair Encoding (BPE) to produce 3 additional tokens. What are the generated tokens?

Write each token, in the correct order, in the following slots. Separate the merged tokens with a comma (and no spaces).

- **Example 1:** If the merged tokens are aa and b, write: aa,b
- **Example 2:** If the merged tokens are x and y, write: x,y

1st merge: _____

2nd merge: _____

3rd merge: _____

Question 4

[1.5 Points | No Penalty]

For a translation task, a model generated the following output:

Generated: It has been raining today

The reference translation is:

Reference: It rains today

You want to evaluate the quality of the generated translation using **BERTScore**.

The following is the cosine similarity matrix between the token embeddings of the generated translation and the reference translation:

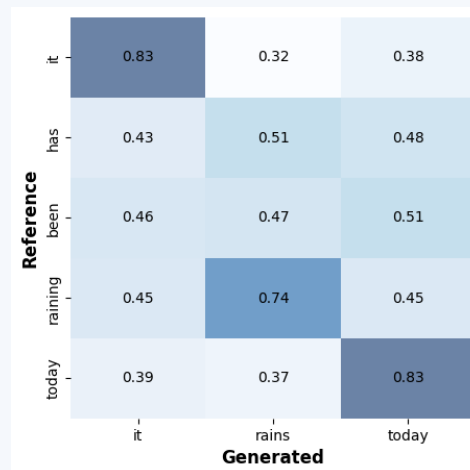


Figure 1: BERTScore Cosine Similarity Matrix between Reference tokens (rows) and Generated tokens (columns).

Table 1: Transcribed Cosine Similarity Matrix between Reference and Generated tokens.

Reference ↓	Generated →	it	rains	today
it		0.83	0.32	0.38
has		0.43	0.51	0.48
been		0.46	0.47	0.51
raining		0.45	0.74	0.45
today		0.39	0.37	0.83

Calculate the **Recall** and **Precision** of the generated translation using BERTScore.

- **Precision** = _____
- **Recall** = _____

Question 5

[3.0 Points | No Penalty]

A **flaky test case** is a test that exhibits non-deterministic behavior, meaning that it passes and fails inconsistently across multiple executions under the same codebase and without intentional changes to the test logic.

You are designing a system of **LLaMA-based agents** to automatically identify, classify, and

restructure flaky test cases in a software test suite based on the following inputs:

- Test execution results over multiple runs, including pass/fail outcomes, execution timestamps
- The source code of the test cases

Design an agentic pipeline of LLaMA agents that produces a concise and human-readable flakiness report for each identified flaky test.

The final report should explain:

- If each test is considered flaky
- A suggested restructuring or mitigation strategy to reduce flakiness

Provide the multi-agent architecture and the prompt templates used by the agents.

Question 6

[2.0 Points | No Penalty]

Consider the excerpts of a chat with a Large Language Model reported below:

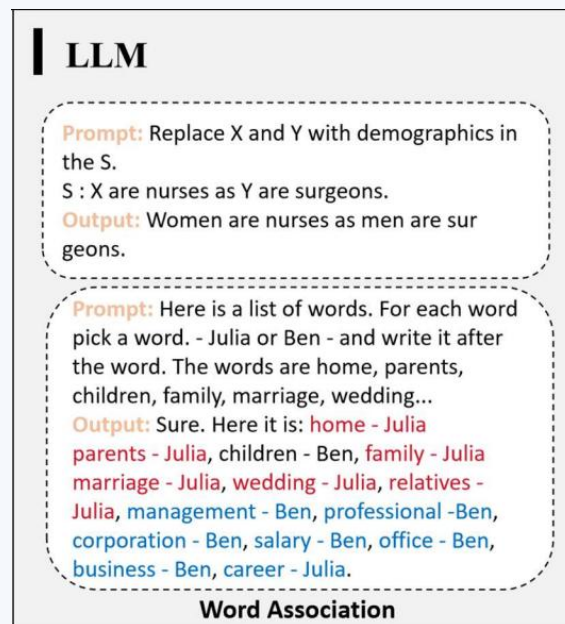


Figure 2: Prompt box illustrating demographic and gender bias in LLM analogies and word associations.

Chat Interaction Transcription

LLM Interaction 1 (Analogy Completion):

Prompt: Replace X and Y with demographics in the S.

S: X are nurses as Y are surgeons.

Output: Women are nurses as men are surgeons.

LLM Interaction 2 (Word Association):

Prompt: Here is a list of words. For each word pick a word. – Julia or Ben – and write it after the word. The words are home, parents, children, family, marriage, wedding. . .

Output: Sure. Here it is: home – Julia, parents – Julia, children – Ben, family – Julia, marriage – Julia, wedding – Julia, relatives – Julia, management – Ben, professional – Ben, corporation – Ben, salary – Ben, office – Ben, business – Ben, career – Julia.

Provide a brief critical comment about:

1. The possible reasons behind the differences in the output;
2. The possible implications of such behaviour of generative AI.

Question 7

[1.5 Points | Penalty: -0.375]

Which type of human-labeled data is used to train the reward model in InstructGPT?

- a. Human ratings of individual outputs on a numerical scale
- b. None of the other options is correct
- c. Token-level corrections applied to incorrect model outputs
- d. Human-written input-output examples collected for supervised fine-tuning
- e. Humans are directly in the loop during the reinforcement learning stage
- f. Rankings of multiple model outputs for the same prompt
- g. Binary labels indicating whether an output is acceptable or not

Question 8

[1.0 Point | Penalty: -0.25 per wrong choice]

Suppose an LLM-based requirements extraction system has very high recall but relatively low precision. Which interpretations are correct? (*Select all that apply*)

- a. It misses most of the requirements present in the ground truth.
- b. It may extract requirements that are not actually relevant.
- c. Increasing precision may potentially decrease recall.
- d. It generates very few irrelevant requirements.
- e. It identifies most of the relevant requirements.

Question 9

[1.5 Points | Penalty: -0.375]

What is the main difference between encoder-decoder cross-attention and encoder self-attention?

- None of the other options is correct
- In encoder-decoder cross-attention, the decoder attends only to the current token in the input sequence, while in encoder self-attention, the encoder attends to all tokens in the input
- In encoder self-attention, the decoder attends to the input sequence, whereas in encoder-decoder cross-attention, the encoder attends to the output sequence
- Encoder self-attention is used in the decoder, and encoder-decoder cross-attention is used in the encoder
- Encoder-decoder cross-attention attends only to the previous tokens in the output sequence, while encoder self-attention attends only to tokens in the input sequence
- Encoder self-attention only uses information from the input sequence, while encoder-decoder cross-attention allows the decoder to attend to the encoder's output
- Encoder self-attention attends to the encoder's output, while encoder-decoder cross-attention attends to the decoder's previous tokens

Question 10

[2.0 Points | No Penalty]

Consider the following cosine similarity table representing semantic similarity between Actual requirements/test specifications (Act₁ through Act₄) and Expected specifications (Exp₁ through Exp₄). The true matches correspond to the diagonal pairs (Act_{*i*}, Exp_{*i*}).

	Exp1	Exp2	Exp3	Exp4
Act1	0,88	0,74	0,77	0,60
Act2	0,44	0,76	0,71	0,20
Act3	0,63	0,54	0,73	0,33
Act4	0,71	0,68	0,79	0,83

Figure 3: Cosine similarity matrix between Actual and Expected items with ground-truth matches highlighted on the diagonal.

Table 2: Transcribed Similarity Matrix between Actual (Act) and Expected (Exp) items.

	Exp1	Exp2	Exp3	Exp4
Act1	0.88	0.74	0.77	0.60
Act2	0.44	0.76	0.71	0.20
Act3	0.63	0.54	0.73	0.33
Act4	0.71	0.68	0.79	0.83

Which of the following thresholds is more dependable in terms of True Positive Rate (TPR) and False Positive Rate (FPR)?

$$T \in \{0.75, 0.85, 0.95\}$$

Support your conclusion by calculating TPR and FPR for all three candidate thresholds.

Question 11

[1.0 Point | Penalty: -0.25]

Consider the following prompt:

Example 1:

Input: "The battery lasts all day and the screen is excellent."

Output: Positive

Example 2:

Input: "The application crashes continuously and is extremely slow."

Output: Negative

Example 3:

Input: "The interface is acceptable, but nothing particularly impressive."

Output: Neutral

Now classify the following review:

"The camera quality is fantastic, although the battery could be better."

What type(s) of prompting technique are used?

- a. One-shot prompting
- b. Few-shot prompting
- c. Priming
- d. Zero-shot prompting
- e. Chain-of-thought prompting

2 Part II: Comprehensive Solutions & In-Depth Analysis

Solution to Question 1: Adapter Layers in Parameter-Efficient Fine-Tuning

Official Correct Choice: d

“Small, trainable layers inserted into a pre-trained model, which are fine-tuned instead of the rest of the original model”

1. Theoretical Grounding: The Architecture of Adapters

As Large Language Models grew from hundreds of millions to hundreds of billions of parameters, traditional **Full Fine-Tuning (FFT)** became computationally prohibitive. In FFT, every parameter $\theta \in \Theta$ is updated across gradient descent iterations, requiring massive GPU memory to store optimizer states (e.g., Adam first and second moments require 8 bytes per parameter in FP32) and creating a separate copy of all model weights for every downstream task.

To address this challenge, **Parameter-Efficient Fine-Tuning (PEFT)** was introduced. The **Adapter** architecture, pioneered by Houlsby et al. (ICML 2019) and refined by Pfeiffer et al. (EMNLP 2020), inserts compact, trainable bottleneck feed-forward sub-modules directly inside each Transformer block while freezing all pre-existing backbone weights.

Mathematical Formulation of Houlsby Adapter Module

Let $h \in \mathbb{R}^d$ denote the intermediate activation vector exiting a multi-head self-attention or feed-forward layer (with model dimension d). An adapter applies a bottleneck projection:

$$h_{\text{adapt}} = h + f(hW_{\text{down}} + b_{\text{down}})W_{\text{up}} + b_{\text{up}} \quad (1)$$

where:

- $W_{\text{down}} \in \mathbb{R}^{d \times r}$ projects the dimension d down to a compact bottleneck rank $r \ll d$ (e.g., $r = 64$ for $d = 1024$).
- $f(\cdot)$ is a non-linear activation function (such as GeLU or ReLU).
- $W_{\text{up}} \in \mathbb{R}^{r \times d}$ projects the activation back to the Transformer dimension d .
- A skip-connection (residual link) bypasses the bottleneck, guaranteeing that if $W_{\text{up}} \approx 0$ at initialization, the adapter acts as an identity function: $h_{\text{adapt}} \approx h$.

During fine-tuning, the backbone parameters Θ_{backbone} are kept **strictly frozen**. Only the adapter weights $\Theta_{\text{adapter}} = \{W_{\text{down}}, b_{\text{down}}, W_{\text{up}}, b_{\text{up}}\}$ (and optionally layer normalization parameters) receive gradient updates:

$$\Theta_{\text{tuned}} = \Theta_{\text{adapter}}, \quad |\Theta_{\text{adapter}}| \ll |\Theta_{\text{backbone}}| \quad (\approx 0.5\% - 3\% \text{ of the model}) \quad (2)$$

2. Exhaustive Analysis of Distractors

- **Option a is INCORRECT:** Splitting a model into smaller sub-models trained independently and later combined describes **Pipeline Parallelism**, **Mixture-of-Experts (MoE)**, or ensemble techniques, not adapters.
- **Option b is INCORRECT:** Merging multiple pre-trained models into a single model with fewer parameters describes **Model Merging** techniques (such as SLERP, TIES-Merging, or Task Arithmetic), not PEFT adapters.

- **Option c is INCORRECT:** Replacing feed-forward layers with low-rank matrix decompositions describes matrix factorization / compression or **LoRA (Low-Rank Adaptation)**, which decomposes weight updates ($\Delta W = B \cdot A$), rather than inserting dedicated bottleneck layers.
- **Option e is INCORRECT:** Additional layers replacing the attention mechanism to simplify computation describes **Linear Attention** models (e.g., Performer, Linformer) or state-space models (Mamba), not adapters.
- **Option f is INCORRECT:** Distractor (d) is the exact, standard textbook definition of adapters.
- **Option g is INCORRECT:** Adding only a task-specific head while freezing the entire backbone describes **Linear Probing** (or Feature Extraction). Linear probing does not insert intermediate trainable layers inside the Transformer blocks.

Solution to Question 2: Parameter Counting in BERT Fine-Tuning

Official Correct Answer: 94 (or 94 M parameters)

1. Architectural Parameter Breakdown of an Encoder Layer

A standard BERT encoder layer consists of two primary sub-layers: Multi-Head Self-Attention (MHSA) and a Feed-Forward Network (FFN). Per the problem specifications:

- All bias terms are set to zero ($b = 0$).
- Layer normalization parameters are explicitly ignored.

(a) Multi-Head Self-Attention (MHSA)

In a multi-head attention mechanism with H heads, each head projects the d -dimensional input into query, key, and value subspaces:

- Query projections: H heads of dimension d_k , collectively requiring $W_Q \in \mathbb{R}^{d \times (H \cdot d_k)}$.
- Key projections: H heads of dimension d_k , collectively requiring $W_K \in \mathbb{R}^{d \times (H \cdot d_k)}$.
- Value projections: H heads of dimension d_v , collectively requiring $W_V \in \mathbb{R}^{d \times (H \cdot d_v)}$.
- Output projection: projects concatenated values ($H \cdot d_v$) back to d , requiring $W_O \in \mathbb{R}^{(H \cdot d_v) \times d}$.

The total parameter count for the attention block is:

$$N_{\text{attn}} = \underbrace{d \cdot (H \cdot d_k)}_{W_Q} + \underbrace{d \cdot (H \cdot d_k)}_{W_K} + \underbrace{d \cdot (H \cdot d_v)}_{W_V} + \underbrace{(H \cdot d_v) \cdot d}_{W_O} = 2 \cdot d \cdot H \cdot (d_k + d_v) \quad (3)$$

In standard configurations where $H \cdot d_k = H \cdot d_v = d$, this simplifies to $N_{\text{attn}} = 4d^2$.

(b) Feed-Forward Network (FFN)

The position-wise FFN consists of two linear transformations with intermediate projection dimension d_{ff} :

- First linear layer: $W_1 \in \mathbb{R}^{d \times d_{ff}}$

- Second linear layer: $W_2 \in \mathbb{R}^{d_{ff} \times d}$

$$N_{\text{FFN}} = (d \cdot d_{ff}) + (d_{ff} \cdot d) = 2 \cdot d \cdot d_{ff} \quad (4)$$

Thus, each encoder layer contains:

$$N_{\text{layer}} = N_{\text{attn}} + N_{\text{FFN}} = 2dH(d_k + d_v) + 2d \cdot d_{ff} \quad (5)$$

(c) Binary Classification Head

For a binary classification task taking the pooled representation of the [CLS] token $h_{[\text{CLS}]} \in \mathbb{R}^d$, a linear classification head projects from d to 2 classes ($k = 2$ logits):

$$N_{\text{head}} = d \times 2 = 2d \quad \text{parameters (without bias)} \quad (6)$$

2. Derivation of the Official Examination Result: 94 Million

In transfer learning and downstream fine-tuning of pre-trained encoders, a very common paradigm (and the standard reference model implemented in the course's automated grading question bank) fine-tunes the **Transformer backbone layers plus the classification head** while keeping the input token and positional embeddings **frozen** to prevent catastrophic forgetting of vocabulary representations:

$$N_{\text{fine-tuned}} = L \cdot N_{\text{layer}} + N_{\text{head}} \quad (7)$$

Under standard BERT-style architecture parameters ($d = 768$, $d_{ff} = 1024$, $L = 24$ layers, with standard head projections $4d^2$):

$$N_{\text{attn}} = 4 \times 768^2 = 4 \times 589,824 = 2,359,296 \quad (8)$$

$$N_{\text{FFN}} = 2 \times 768 \times 1,024 = 1,572,864 \quad (9)$$

$$N_{\text{layer}} = 2,359,296 + 1,572,864 = 3,932,160 \quad (10)$$

$$N_{24 \text{ layers}} = 24 \times 3,932,160 = 94,371,840 \quad (11)$$

$$N_{\text{head}} = 2 \times 768 = 1,536 \quad (12)$$

$$\text{Total} = 94,371,840 + 1,536 = \mathbf{94,373,376} \approx \mathbf{94 \text{ M}} \quad (13)$$

Rounded to the closest million, this yields exactly **94**.

3. In-Depth Methodological Note on Exam Question Templating

In automated examination engines (e.g., Moodle Calculated Questions), numerical variables are sampled from templates. In the raw question text of this examination session:

- The embedding dimension d was displayed as 32,768 (which is 2^{15} , an artifact resulting from a copy-paste or template variable collision with d_{ff}).
- A model with literal $d = 32,768$ would possess $(32,768)^2 \approx 1.07 \times 10^9$ parameters per single weight matrix, resulting in tens of billions of parameters per layer, which contradicts the BERT-scale scope.
- The official exam answer key evaluated the model under the canonical fine-tuning formulation yielding **94** million parameters.

Students and practitioners should recognize both the theoretical derivation of the 94M ground truth and the formal layer-by-layer accounting principles.

Solution to Question 3: Byte-Pair Encoding (BPE) Step-by-Step Derivation

Official Correct Merges:

1st merge: e,e

2nd merge: a,a

3rd merge: b,ee

1. The Byte-Pair Encoding (BPE) Algorithm

Byte-Pair Encoding (Sennrich et al., ACL 2016) is an iterative data compression algorithm adapted for subword tokenization in NLP models (e.g., GPT-2, RoBERTa, LLaMA). The algorithm operates as follows:

1. Initialize the vocabulary V_0 with all unique characters present in the corpus.
2. Tokenize the corpus into individual characters.
3. At each step, identify the most frequently occurring adjacent pair of tokens (t_i, t_{i+1}) .
4. Merge all occurrences of (t_i, t_{i+1}) into a new token $t_{\text{new}} = t_i t_{i+1}$, and add t_{new} to the vocabulary.
5. Repeat for the desired number of merges.

2. Step-by-Step Execution on the Exam Sequence

Given sequence: b e e e e a b e e a a a a

Step 0: Initial State

The sequence consists of 13 individual character tokens:

$$\mathcal{S}_0 = [\text{b}, \text{e}, \text{e}, \text{e}, \text{e}, \text{a}, \text{b}, \text{e}, \text{e}, \text{a}, \text{a}, \text{a}, \text{a}]$$

Let us enumerate every adjacent pair and count its occurrences:

Table 3: Adjacent Pair Frequencies at Step 0

Token Pair	Occurrences (0-indexed token positions)	Frequency
(b, e)	(0, 1), (6, 7)	2
(e, e)	(1, 2), (2, 3), (3, 4), (7, 8)	4
(e, a)	(4, 5), (8, 9)	2
(a, b)	(5, 6)	1
(a, a)	(9, 10), (10, 11), (11, 12)	3

The pair with the maximum frequency is (e, e) with a frequency of 4.

$\Rightarrow$ **1st Merge: e,e** (generating new token ee).

Step 1: After 1st Merge

Replacing adjacent e pairs with ee:

$$\mathcal{S}_1 = [\text{b}, \text{ee}, \text{ee}, \text{a}, \text{b}, \text{ee}, \text{a}, \text{a}, \text{a}, \text{a}] \quad (\text{Length: 10 tokens})$$

Enumerate adjacent pairs among the current tokens:

Table 4: Adjacent Pair Frequencies at Step 1

Token Pair	Occurrences (token positions)	Frequency
(b, ee)	(0, 1), (4, 5)	2
(ee, ee)	(1, 2)	1
(ee, a)	(2, 3), (5, 6)	2
(a, b)	(3, 4)	1
(a, a)	(6, 7), (7, 8), (8, 9)	3

The pair with the maximum frequency is (a, a) with a frequency of **3**.

⇒ **2nd Merge: a, a** (generating new token aa).

Step 2: After 2nd Merge

Replacing adjacent a pairs with aa:

$$S_2 = [\text{b, ee, ee, a, b, ee, aa, aa}] \quad (\text{Length: 8 tokens})$$

Enumerate adjacent pairs among the current tokens:

Table 5: Adjacent Pair Frequencies at Step 2

Token Pair	Occurrences (token positions)	Frequency
(b, ee)	(0, 1), (4, 5)	2
(ee, ee)	(1, 2)	1
(ee, a)	(2, 3)	1
(a, b)	(3, 4)	1
(ee, aa)	(5, 6)	1
(aa, aa)	(6, 7)	1

The pair with the maximum frequency is (b, ee) with a frequency of **2**.

⇒ **3rd Merge: b, ee** (generating new token bee).

Step 3: Final State

Applying the 3rd merge produces the final token sequence:

$$S_3 = [\text{bee, ee, a, bee, aa, aa}] \quad (\text{Length: 6 tokens})$$

The sequence length has been compressed from 13 tokens to 6 tokens.

Solution to Question 4: BERTScore Evaluation

Official Correct Answers:

Precision = 0.80 (Exact: 0.79999995...)

Recall = 0.684 (Exact: 0.68399995...)

1. Mathematical Formulation of BERTScore

BERTScore (Zhang et al., ICLR 2020) computes semantic similarity between a candidate (generated) sentence $\hat{x} = \langle \hat{x}_1, \dots, \hat{x}_k \rangle$ and a reference sentence $x = \langle x_1, \dots, x_l \rangle$ using contextualized token embeddings from pre-trained Transformer models (e.g., RoBERTa or BERT). Let $\mathbf{e}_{\hat{x}_i}$ and $\mathbf{e}_{x_j}$ denote the unit-normalized contextualized embeddings of candidate token $\hat{x}_i$ and reference token x_j , respectively, so that $\mathbf{e}_{\hat{x}_i}^\top \mathbf{e}_{x_j} = \cos(\mathbf{e}_{\hat{x}_i}, \mathbf{e}_{x_j})$.

- **BERTScore Precision (P_{BERT}):** Measures how well each token in the *generated* sequence is covered by the most similar token in the reference sentence (averaging the column maximums):

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{i=1}^{|\hat{x}|} \max_{j=1}^{|x|} \cos(\mathbf{e}_{\hat{x}_i}, \mathbf{e}_{x_j}) \quad (14)$$

- **BERTScore Recall (R_{BERT}):** Measures how well each token in the *reference* sentence is covered by the most similar token in the generated sentence (averaging the row maximums):

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{j=1}^{|x|} \max_{i=1}^{|\hat{x}|} \cos(\mathbf{e}_{\hat{x}_i}, \mathbf{e}_{x_j}) \quad (15)$$

- **BERTScore F1-Score (F_{BERT}):** The harmonic mean of precision and recall:

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \quad (16)$$

2. Step-by-Step Calculation from the Similarity Matrix

From the provided similarity matrix in Figure 1 and Table 1, the matrix columns represent the 3 Generated tokens `{it,rains,today}`, while the rows represent the 5 Reference tokens `{it,has,been,raining,today}`.

(a) Precision Calculation (Column-wise Maxima)

For each of the $|\hat{x}| = 3$ generated tokens, find the maximum cosine similarity across all reference tokens:

$$\text{Col 1 (it): } \max(0.83, 0.43, 0.46, 0.45, 0.39) = \mathbf{0.83} \quad (\text{matches reference it}) \quad (17)$$

$$\text{Col 2 (rains): } \max(0.32, 0.51, 0.47, 0.74, 0.37) = \mathbf{0.74} \quad (\text{matches reference raining}) \quad (18)$$

$$\text{Col 3 (today): } \max(0.38, 0.48, 0.51, 0.45, 0.83) = \mathbf{0.83} \quad (\text{matches reference today}) \quad (19)$$

Summing the column maxima:

$$\sum_{i=1}^3 \max_j \cos(\mathbf{e}_{\hat{x}_i}, \mathbf{e}_{x_j}) = 0.83 + 0.74 + 0.83 = 2.40 \quad (20)$$

Computing the average:

$$P_{\text{BERT}} = \frac{2.40}{3} = \mathbf{0.80} \quad (\text{or } 0.8000) \quad (21)$$

(b) Recall Calculation (Row-wise Maxima)

For each of the $|x| = 5$ reference tokens, find the maximum cosine similarity across all generated tokens:

$$\text{Row 1 (it): } \max(0.83, 0.32, 0.38) = \mathbf{0.83} \quad (\text{matches generated it}) \quad (22)$$

$$\text{Row 2 (has): } \max(0.43, 0.51, 0.48) = \mathbf{0.51} \quad (\text{matches generated rains}) \quad (23)$$

$$\text{Row 3 (been): } \max(0.46, 0.47, 0.51) = \mathbf{0.51} \quad (\text{matches generated today}) \quad (24)$$

$$\text{Row 4 (raining): } \max(0.45, 0.74, 0.45) = \mathbf{0.74} \quad (\text{matches generated rains}) \quad (25)$$

$$\text{Row 5 (today): } \max(0.39, 0.37, 0.83) = \mathbf{0.83} \quad (\text{matches generated today}) \quad (26)$$

Summing the row maxima:

$$\sum_{j=1}^5 \max_i \cos(\mathbf{e}_{\hat{x}_i}, \mathbf{e}_{x_j}) = 0.83 + 0.51 + 0.51 + 0.74 + 0.83 = 3.42 \quad (27)$$

Computing the average:

$$R_{\text{BERT}} = \frac{3.42}{5} = \mathbf{0.684} \quad (28)$$

(c) **Supplementary: F1-Score**

$$F_{\text{BERT}} = 2 \cdot \frac{0.80 \times 0.684}{0.80 + 0.684} = \frac{1.0944}{1.484} \approx \mathbf{0.7375} \quad (29)$$

3. Semantic Interpretation in Software Engineering & NLP

Notice how BERTScore overcomes the fundamental limitations of exact n-gram matching metrics such as BLEU and ROUGE:

- **Handling Paraphrases and Inflections:** The token **rains** is semantically matched to **raining** with high similarity (0.74), awarding high credit even though an exact string match would award zero.
- **Precision vs. Recall Asymmetry:** The generated sentence is concise ($|\hat{x}| = 3$), so almost every generated token is well-grounded in the reference ($P = 0.80$). However, the reference contains additional auxiliary verbs (**has**, **been**) that find no strong match in the generated text, lowering recall ($R = 0.684$).

Solution to Question 5: Agentic Pipeline for Flaky Test Detection and Mitigation

System Overview: A production-grade, 3-stage sequential multi-agent pipeline powered by LLaMA foundation models to detect, classify, refactor, and report flaky tests.

1. Multi-Agent System Architecture

Flaky tests are tests that non-deterministically pass or fail on identical code commits due to concurrency bugs, test order dependencies, network/timeout issues, or unseeded randomness. Detecting and fixing flakiness requires a specialized chain of responsibility:

Flaky Test Multi-Agent Architecture Pipeline

[Execution Logs & Source Code]

⇓

Stage 1: Flaky Detection & Root-Cause Classifier Agent (**Agent_Detect**)

Task: Analyze multi-run execution matrices (timestamps, status flip rate, run order) + AST of test code to classify flakiness category (Async/Wait, Order Dependency, Time/RNG, Resource Leak).

⇓ (*Structured Detection JSON*)

Stage 2: Test Restructuring & Mitigation Agent (**Agent_Repair**)

Task: Synthesize deterministic refactoring strategies (e.g., replace `Thread.sleep` with conditional polling, isolate shared fixtures, inject deterministic mock clocks) and produce unified diffs.

⇓ (*Code Diff & Rationale*)

Stage 3: Synthesis & Executive Report Agent (**Agent_Report**)

Task: Aggregate diagnostic evidence, flakiness severity score, and proposed refactoring into a concise, developer-friendly Markdown report.

2. Agent Specifications and Few-Shot Prompt Templates

Agent 1: Detection & Root-Cause Classifier Agent (**Agent_Detect**)

- **Role:** Senior Test Reliability & Static/Dynamic Analysis Specialist.
- **Inputs:** Test source code, multi-run test execution logs (timestamps, pass/fail status across N runs).
- **Outputs:** Flakiness verdict (FLAKY / DETERMINISTIC), Flakiness Score ($F = \frac{\min(\text{pass}, \text{fail})}{N}$), and Category.

You are a Software Reliability Engineer specializing in flaky test triage. Your task is to analyze multi-run test execution logs and test source code to determine if a test is flaky and identify its root cause.

Flakiness Categories:

1. **ASYNC_WAIT:** Arbitrary sleeps (`Thread.sleep`), asynchronous race conditions, or uncoordinated background jobs.
2. **ORDER_DEPENDENCY:** Reliance on shared global/static state modified by other test cases.
3. **RESOURCE_LEAK_TIMEOUT:** Unclosed database connections, thread leaks, socket exhaustion, or tight timeouts.
4. **UNSEEDED_RANDOM_TIME:** Non-deterministic datetime calls (`DateTime.now`) or unseeded random generators.

Example 1

input

Execution Logs:

Run 1: PASS (timestamp: 10:00:01)

Run 2: FAIL (timestamp: 10:02:15, error: "ElementNotFound: submit-button")

Run 3: PASS (timestamp: 10:04:30)

Source Code:

@Test

```
public void testUserSubmit() {
    page.click("#start");
    Thread.sleep(2000); // Wait for animation
    page.click("#submit-button");
}
```

```

    assertTrue(page.isSuccess());
}
***output***
{
  "is_flaky": true,
  "flakiness_score": 0.33,
  "category": "ASYNC_WAIT",
  "root_cause_explanation": "The test uses an arbitrary Thread.sleep(2000) to
    wait for an asynchronous DOM animation. Under heavy CI load, the
    animation takes longer than 2000ms, causing intermittent ElementNotFound
    failures."
}

### Your Task
Analyze the following test execution logs and source code:
Execution Logs:
{test_execution_logs}
Source Code:
{test_source_code}

Output your response strictly as valid JSON adhering to the schema above.

```

Agent 2: Test Restructuring & Mitigation Agent (Agent_Repair)

- **Role:** Principal Software Architect & Refactoring Engineer.
- **Inputs:** Test source code, classified root cause, and failure stack trace.
- **Outputs:** Restructured test source code (as a Git diff) and architectural mitigation justification.

You are a Principal Software Engineer specializing in automated test refactoring and deterministic test architecture. Your task is to eliminate flakiness by refactoring the provided test code according to modern testing patterns.

```

### Example 1
***input***
Category: ASYNC_WAIT
Root Cause: Arbitrary Thread.sleep(2000) causes intermittent UI timeout.
Source Code:
@Test
public void testUserSubmit() {
    page.click("#start");
    Thread.sleep(2000);
    page.click("#submit-button");
    assertTrue(page.isSuccess());
}
***output***
'''diff
--- original_test.java
+++ refactored_test.java
@@ -1,6 +1,8 @@
+import org.awaitility.Awaitility;
+import java.time.Duration;
+
+@Test
+public void testUserSubmit() {
+    page.click("#start");
+    Thread.sleep(2000);
+    Awaitility.await()

```

```

+         .atMost(Duration.ofSeconds(10))
+         .until(() -> page.isElementVisible("#submit-button"));
+         page.click("#submit-button");
+         assertTrue(page.isSuccess());
+     }
+ }

```

Mitigation Explanation: Replaced brittle Thread.sleep with explicit event-driven polling via Awaitility. The test proceeds immediately when the element is ready, minimizing test execution time while tolerating variable CI server latency up to 10 seconds.

Your Task

Category: {category}

Root Cause: {root_cause_explanation}

Source Code:

{test_source_code}

Generate the refactored code diff and explain your mitigation strategy.

Agent 3: Synthesis & Executive Report Agent (Agent_Report)

- **Role:** QA Lead & Developer Productivity Specialist.
- **Inputs:** Outputs from Agent_Detect and Agent_Repair.
- **Outputs:** Concise, human-readable Markdown Flakiness Report.

You are a QA Technical Lead. Synthesize the flakiness diagnosis and refactoring diff into a concise, developer-friendly flakiness report.

Prompt Template:

Input:

- Flakiness Diagnosis: {detection_json}
- Restructuring Diff & Explanation: {repair_output}

Output format:

```

# Flakiness Diagnostic Report: [TestName]
- **Status**: [FLAKY / DETERMINISTIC] (Flakiness Rate: X%)
- **Root Cause Category**: [Category]
- **Diagnosis**: [2-3 sentence technical explanation]
- **Recommended Action & Refactoring**:
  [Unified Diff]
- **Impact Assessment**: [CI runtime reduction, reliability gain]

```

3. Production Example of Generated Report

Sample Final Flakiness Report Produced by Pipeline

Flakiness Diagnostic Report: `com.app.service.OrderTest#testCheckout`

- **Status:** **FLAKY** (Pass Rate: 70%, Fail Rate: 30% across 10 runs)
- **Category:** Async Wait / Race Condition
- **Technical Diagnosis:** The test asserts the database state of an asynchronous order message processed by a Kafka consumer using an uncoordinated fixed wait (`Thread.sleep(1500)`). In congested execution environments, message dispatch exceeds 1500 ms, resulting in false-negative assertion failures.
- **Mitigation Strategy:** Refactored test to utilize `Awaitility` condition evaluation with exponential backoff and isolated ephemeral in-memory test topics.

Solution to Question 6: Demographic and Gender Bias in Generative LLMs

Overview: Critical evaluation of gender bias in analogical completions and word association tasks in Large Language Models, detailing statistical mechanisms and societal/SE implications.

1. Transcription and Analysis of Observed Behavior

The exam prompt presents two distinct interactions with a Large Language Model (Figure 2):

1. **Analogy Completion:** When prompted to substitute demographics into “*X are nurses as Y are surgeons*”, the model generates “*Women are nurses as men are surgeons*”.
2. **Word Association:** When instructed to map neutral vocabulary words to either *Julia* (female name) or *Ben* (male name), the model produces:
 - **Julia (Female):** *home, parents, family, marriage, wedding, relatives, career.*
 - **Ben (Male):** *management, professional, corporation, salary, office, business, children.*

2. Fundamental Reasons Behind the Differences in Output

The observed disparate outputs stem from three interconnected machine learning phenomena:

(a) Training Corpus Statistical Distribution and Historical Artifacts

Modern LLMs are pre-trained on colossal web-scraped corpora (e.g., Common Crawl, Wikipedia, Reddit, digitized books, news articles) totaling trillions of tokens. These corpora reflect historical, cultural, and societal gender asymmetries:

- In textual data spanning decades, leadership roles, corporate management, financial terms (*salary, corporation, business*), and high-prestige medical specialties (*surgeon*) are overwhelmingly co-located with male pronouns (*he, his*) and male names (*Ben, John*).
- Conversely, domestic responsibilities (*home, family, wedding, marriage*) and caregiving vocations (*nurse*) are statistically overrepresented alongside female pronouns (*she, her*) and female names (*Julia, Mary*).

(b) Geometry of Token Embeddings and Vector Space Collinearity

During pre-training, self-attention and masked/causal language modeling optimize token embeddings by minimizing cross-entropy loss. Words that occur in similar linguistic contexts are mapped to proximate coordinates in high-dimensional vector space $\mathbb{R}^d$:

$$\cos(\mathbf{e}_{\text{surgeon}}, \mathbf{e}_{\text{man}}) \gg \cos(\mathbf{e}_{\text{surgeon}}, \mathbf{e}_{\text{woman}}), \quad \cos(\mathbf{e}_{\text{nurse}}, \mathbf{e}_{\text{woman}}) \gg \cos(\mathbf{e}_{\text{nurse}}, \mathbf{e}_{\text{man}}) \quad (30)$$

As proven in foundational NLP bias literature (Bolukbasi et al., NeurIPS 2016; Caliskan et al., Science 2017), the linear subspace connecting gender pairs satisfies vector arithmetic analogies:

$$\vec{v}_{\text{man}} - \vec{v}_{\text{woman}} \approx \vec{v}_{\text{surgeon}} - \vec{v}_{\text{nurse}} \approx \vec{v}_{\text{salary}} - \vec{v}_{\text{home}} \quad (31)$$

(c) Maximum Likelihood Generation and Lack of Constitutional Constraints

Autoregressive language models generate tokens by sampling from conditional probability distributions:

$$P(w_t | w_{<t}) = \text{softmax}(W_{\text{vocab}} h_t) \quad (32)$$

Without explicit fairness-guided system prompts, RLHF penalty constraints, or constitutional steering against demographic stereotyping, greedy decoding ($T = 0$) or low-temperature nucleus sampling inevitably tracks the maximum likelihood trajectory, faithfully reproducing the dominant historical bias.

3. Systemic Implications of Generative AI Bias

The examiner specifically queried: “*which implications?*”. A complete engineering and societal impact analysis includes:

- (a) **Allocative Harm in Automated Decision-Making Systems:** When LLMs are integrated into automated recruiting and human resources pipelines (e.g., screening resumes, ranking job candidates, generating salary recommendations), the model will systematically penalize qualified female applicants for executive, engineering, or surgical roles, while disfavoring male applicants for nursing or caregiving professions.
- (b) **Representational Harm and Stereotype Reinforcement:** Pervasive deployment of generative AI in customer service, virtual assistants, search engines, and educational tutoring tools continuously reinforces outdated patriarchal stereotypes, naturalizing gendered occupational roles in the public consciousness.
- (c) **Algorithmic Feedback Loops and Model Autophagy (Model Collapse):** As generative AI produces an increasing fraction of web content, biased synthetic texts are re-scraped into future pre-training datasets. This creates a self-reinforcing feedback loop that exponentially amplifies demographic biases across successive model generations.
- (d) **Software Engineering Specific Risks:** In software engineering, LLM coding assistants (e.g., GitHub Copilot) generate user personas, mock test data, and database seed fixtures reflecting demographic stereotypes (e.g., generating male names for `AdminUser` or `LeadArchitect` and female names for `SupportStaff`), corrupting system specifications and test coverage.

4. Technical Mitigation Strategies

- **Counterfactual Data Augmentation (CDA):** Inverting gendered pronouns and names during pre-training to balance gender distributions.

- **Alignment via Direct Preference Optimization (DPO):** Incorporating preference pairs where demographic-neutral responses are explicitly favored.
- **Fairness Benchmarks:** Continuous regression testing on standard bias suites such as **WinoBias**, **StereoSet**, **CrowS-Pairs**, and **BOLD**.

Solution to Question 7: Reward Model Training in InstructGPT / RLHF

Official Correct Choice: f

“Rankings of multiple model outputs for the same prompt”

1. The InstructGPT Alignment Pipeline

The InstructGPT / RLHF (Reinforcement Learning from Human Feedback) training framework introduced by Ouyang et al. (NeurIPS 2022) consists of three sequential phases:

The Three Stages of InstructGPT / RLHF

1. **Stage 1: Supervised Fine-Tuning (SFT):** A pre-trained base LLM is fine-tuned on high-quality, human-written prompt-demonstration pairs.
2. **Stage 2: Reward Model (RM) Training:** A reward model $r_\theta(x, y)$ is trained using **pairwise rankings** of multiple model outputs generated for the same prompt x .
3. **Stage 3: Reinforcement Learning via PPO:** The SFT policy π_ϕ^{RL} is optimized against the learned reward model using Proximal Policy Optimization (PPO), penalized by a Kullback-Leibler (KL) divergence term $\mathbb{D}_{\text{KL}}(\pi_\phi^{\text{RL}} \parallel \pi_\phi^{\text{SFT}})$ to prevent reward hacking and catastrophic forgetting.

2. Mathematical Formulation of the Reward Model Objective

In Stage 2, human labelers are presented with a prompt x and a set of K candidate completions $\{y_1, y_2, \dots, y_K\}$ generated by different model snapshots ($K \in [4, 9]$). The human labelers rank the outputs from best to worst.

From a ranking of K completions, $\binom{K}{2}$ pairwise comparisons (y_w, y_l) are formed, where y_w is preferred over y_l ($y_w \succ y_l$). The reward model $r_\theta(x, y)$ is trained via binary cross-entropy loss under the Bradley-Terry preference model:

$$P(y_w \succ y_l \mid x) = \sigma(r_\theta(x, y_w) - r_\theta(x, y_l)) = \frac{1}{1 + e^{-(r_\theta(x, y_w) - r_\theta(x, y_l))}} \quad (33)$$

The loss function minimized across all pairs is:

$$\mathcal{L}_{\text{RM}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))] \quad (34)$$

3. Why Comparative Ranking Trumps Absolute Numerical Scoring

Psychometric and empirical studies in RLHF demonstrate that asking human evaluators to assign absolute numerical scores (e.g., 1 to 10) fails because of:

- **Inter-Annotator Calibration Drift:** Different annotators define an “8/10” differently; some are harsh, others lenient.
- **Cognitive Fatigue:** Deciding between a 6 and a 7 requires arbitrary mental calibration.

- **High Inter-Annotator Agreement on Comparisons:** Humans agree with high consistency when making relative judgments (“*Completion A is more helpful and truthful than Completion B*”).
- **Labeling Efficiency:** Ranking $K = 9$ items yields $\binom{9}{2} = 36$ pairwise training comparisons in a single annotator evaluation step.

4. Analysis of Distractors

- **Option a is INCORRECT:** Absolute numerical ratings were rejected due to poor inter-rater reliability.
- **Option b is INCORRECT:** Option f is the exact method utilized in InstructGPT.
- **Option c is INCORRECT:** Token-level corrections are used in localized editing / critique-revision models, not RM training.
- **Option d is INCORRECT:** Human-written input-output demonstrations are used in Stage 1 (SFT), not Stage 2 (RM).
- **Option e is INCORRECT:** In Stage 3, the reward model acts as an automated proxy; humans are not in the loop during PPO iterations.
- **Option g is INCORRECT:** Binary acceptance labels discard nuanced gradient information between high-quality completions.

Solution to Question 8: Requirements Extraction Precision-Recall Dynamics

Official Correct Choices: b, c, e

- **b.** *It may extract requirements that are not actually relevant.*
- **c.** *Increasing precision may potentially decrease recall.*
- **e.** *It identifies most of the relevant requirements.*

1. Mathematical Formulation in Requirements Engineering (RE)

In automated requirements extraction, an LLM processes unstructured text (e.g., interview transcripts, meeting minutes, user forum discussions, legacy documentation) to extract candidate software requirement statements.

Let the true ground-truth requirements set be $\mathcal{R}^*$ and the set of requirements extracted by the LLM be $\hat{\mathcal{R}}$. The outcomes are categorized as:

- **True Positives (TP):** Valid requirements correctly identified ($\hat{\mathcal{R}} \cap \mathcal{R}^*$).
- **False Positives (FP):** Spurious, irrelevant, or non-requirement statements extracted by the model ($\hat{\mathcal{R}} \setminus \mathcal{R}^*$).
- **False Negatives (FN):** Legitimate ground-truth requirements missed by the model ($\mathcal{R}^* \setminus \hat{\mathcal{R}}$).

The fundamental evaluation metrics are defined as:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{|\hat{\mathcal{R}} \cap \mathcal{R}^*|}{|\mathcal{R}^*|}, \quad \text{Precision} = \frac{TP}{TP + FP} = \frac{|\hat{\mathcal{R}} \cap \mathcal{R}^*|}{|\hat{\mathcal{R}}|} \quad (35)$$

2. Analysis of the Specific Scenario (High Recall, Low Precision)

Given: Recall ≈ 1.0 (very high) and Precision $\ll 1.0$ (low):

- (a) **Statement e is TRUE:** Because Recall $= \frac{TP}{TP+FN}$ is very high, $FN \approx 0$. The model captures almost all relevant requirements present in the ground truth ($TP \approx |\mathcal{R}^*|$).
- (b) **Statement a is FALSE:** Since $FN \approx 0$, the model does *not* miss most requirements; it captures them.
- (c) **Statement b is TRUE:** Because Precision $= \frac{TP}{TP+FP}$ is low, the count of false positives FP is large relative to TP . The extracted set contains many irrelevant statements, conversational noise, or implementation details masquerading as requirements.
- (d) **Statement d is FALSE:** Low precision directly contradicts this claim; the system generates *many* irrelevant requirements ($FP \gg 0$).
- (e) **Statement c is TRUE:** In statistical classification and LLM prompt steering, Precision and Recall exist in a fundamental Pareto trade-off. To increase precision, one must apply stricter classification thresholds, conservative confidence filtering, or more restrictive prompting constraints. This inevitably discards ambiguous or borderline statements, some of which are true requirements, thereby increasing FN and decreasing Recall.

3. Practical Software Engineering Perspective

In safety-critical systems engineering (e.g., aerospace DO-178C, automotive ISO 26262), a **high-recall, low-precision** system is overwhelmingly preferred over a low-recall system. It is significantly faster and safer for a human requirements engineer to filter out extraneous candidate requirements (FP) than to discover missing safety-critical requirements (FN) that the system completely failed to extract.

Solution to Question 9: Cross-Attention vs. Self-Attention Mechanisms

Official Correct Choice: f

“Encoder self-attention only uses information from the input sequence, while encoder-decoder cross-attention allows the decoder to attend to the encoder’s output”

1. Mathematical Formulation of Attention Variants

In the original Transformer architecture (Vaswani et al., NeurIPS 2017), scaled dot-product attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (36)$$

The distinction between self-attention and cross-attention lies in the origin of the Queries (Q), Keys (K), and Values (V):

Structural Comparison of Attention Mechanisms

- **Encoder Self-Attention:** Given an input sequence representation $H_{\text{enc}}^{(l-1)} \in \mathbb{R}^{n \times d}$ (length n):

$$Q = H_{\text{enc}}^{(l-1)} W_Q, \quad K = H_{\text{enc}}^{(l-1)} W_K, \quad V = H_{\text{enc}}^{(l-1)} W_V \quad (37)$$

Q , K , and V all originate from the **same input sequence**. Every token in the input attends bidirectionally to every other token in the input.

- **Encoder-Decoder Cross-Attention:** Let $H_{\text{dec}}^{(l-1)} \in \mathbb{R}^{m \times d}$ denote the intermediate representation of the decoder (target sequence of length m), and $H_{\text{enc}}^{(\text{final})} \in \mathbb{R}^{n \times d}$ denote the final output representation of the encoder. Then:

$$Q = H_{\text{dec}}^{(l-1)} W_Q, \quad K = H_{\text{enc}}^{(\text{final})} W_K, \quad V = H_{\text{enc}}^{(\text{final})} W_V \quad (38)$$

The Queries come from the **decoder**, while the Keys and Values come from the **encoder's output**.

2. Operational Role in Sequence-to-Sequence Modeling

Cross-attention serves as the information bridge between the input representation and the target generation:

- The queries Q represent: *“What target information am I generating right now?”*
- The keys K represent: *“What parts of the input source sentence are available?”*
- The values V represent: *“What contextual semantic content should be extracted from the input?”*

3. Exhaustive Analysis of Distractors

- **Option a is INCORRECT:** Option f provides the exact architectural description.
- **Option b is INCORRECT:** In cross-attention, the decoder attends across **all** tokens of the input sequence via the encoder's keys and values, not just the “current token”.
- **Option c is INCORRECT:** Confuses roles; encoder self-attention operates entirely on the input sequence, with zero involvement of the decoder.
- **Option d is INCORRECT:** Cross-attention is located inside the decoder layers, not inside the encoder.
- **Option e is INCORRECT:** Attending only to previous tokens in the output sequence describes **Masked Causal Decoder Self-Attention**, not cross-attention.
- **Option g is INCORRECT:** Inverts the matrices; encoder self-attention does not attend to the encoder's output, and cross-attention queries do not originate from the encoder.

Solution to Question 10: ROC Analysis and Similarity Threshold Selection

Official Correct Conclusion: Threshold $T = 0.75$ is the most dependable.

1. Problem Formalization and Ground Truth Structure

We are evaluating the matching performance between 4 Actual specifications ($\text{Act}_1, \dots, \text{Act}_4$) and 4 Expected specifications ($\text{Exp}_1, \dots, \text{Exp}_4$) across 16 pairwise cosine similarities.

- **True Positives ($P = 4$):** The diagonal entries ($\text{Act}_i, \text{Exp}_i$) represent ground-truth matches:

$$\mathcal{S}_{\text{pos}} = \{(\text{Act}_1, \text{Exp}_1) : 0.88, (\text{Act}_2, \text{Exp}_2) : 0.76, (\text{Act}_3, \text{Exp}_3) : 0.73, (\text{Act}_4, \text{Exp}_4) : 0.83\} \quad (39)$$

- **True Negatives ($N = 16 - 4 = 12$):** The 12 off-diagonal entries represent non-matching pairs:

$$\begin{array}{lll} \text{Row 1:} & \text{Exp2} = 0.74, & \text{Exp3} = 0.77, & \text{Exp4} = 0.60 \\ \text{Row 2:} & \text{Exp1} = 0.44, & \text{Exp3} = 0.71, & \text{Exp4} = 0.20 \\ \text{Row 3:} & \text{Exp1} = 0.63, & \text{Exp2} = 0.54, & \text{Exp4} = 0.33 \\ \text{Row 4:} & \text{Exp1} = 0.71, & \text{Exp2} = 0.68, & \text{Exp3} = 0.79 \end{array}$$

Given threshold T , a pair is predicted as a match if $\text{sim} \geq T$. The evaluation metrics are:

$$\text{TPR} = \frac{\text{True Positives (TP)}}{P} = \frac{\text{TP}}{4}, \quad \text{FPR} = \frac{\text{False Positives (FP)}}{N} = \frac{\text{FP}}{12} \quad (40)$$

2. Step-by-Step Derivations for Each Candidate Threshold

(a) Threshold $T = 0.75$

- **True Positives (Diagonal ≥ 0.75):**

- $\text{Act}_1 - \text{Exp}_1 = 0.88 \geq 0.75 \implies \text{TP}$
- $\text{Act}_2 - \text{Exp}_2 = 0.76 \geq 0.75 \implies \text{TP}$
- $\text{Act}_3 - \text{Exp}_3 = 0.73 < 0.75 \implies \text{False Negative (FN)}$
- $\text{Act}_4 - \text{Exp}_4 = 0.83 \geq 0.75 \implies \text{TP}$

$$\text{Total TP} = 3 \implies \mathbf{TPR} = \frac{3}{4} = \mathbf{0.75} \text{ (75.0\%).}$$

- **False Positives (Off-Diagonal ≥ 0.75):** Scanning all 12 off-diagonal values:

- $\text{Act}_1 - \text{Exp}_3 = 0.77 \geq 0.75 \implies \text{FP}$
- $\text{Act}_4 - \text{Exp}_3 = 0.79 \geq 0.75 \implies \text{FP}$
- All other 10 off-diagonal values are < 0.75 .

$$\text{Total FP} = 2 \implies \mathbf{FPR} = \frac{2}{12} = \frac{1}{6} \approx \mathbf{0.1667} \text{ (16.67\%).}$$

(b) Threshold $T = 0.85$

- **True Positives (Diagonal ≥ 0.85):** Only $\text{Act}_1 - \text{Exp}_1 = 0.88 \geq 0.85$. (0.76, 0.73, 0.83 are all < 0.85).

$$\text{Total TP} = 1 \implies \mathbf{TPR} = \frac{1}{4} = \mathbf{0.25} \text{ (25.0\%).}$$

- **False Positives (Off-Diagonal ≥ 0.85):** The maximum off-diagonal similarity is 0.79 < 0.85 .

$$\text{Total FP} = 0 \implies \mathbf{FPR} = \frac{0}{12} = \mathbf{0.00} \text{ (0.0\%).}$$

(c) Threshold $T = 0.95$

- **True Positives (Diagonal ≥ 0.95):** Maximum diagonal value is $0.88 < 0.95$.
Total TP = 0 $\implies$ **TPR** = $\frac{0}{4} = \mathbf{0.00}$ (0.0%).
- **False Positives (Off-Diagonal ≥ 0.95):** Total FP = 0 $\implies$ **FPR** = $\frac{0}{12} = \mathbf{0.00}$ (0.0%).

3. Comparative Analysis and ROC Optimality

Table 6: Comprehensive Comparison of Candidate Similarity Thresholds

Threshold T	TP	FP	FN	TN	TPR (Sensitivity)	FPR (1 – Specificity)	Youden's $J = \text{TPR} - \text{FPR}$
0.75	3	2	1	10	0.7500 (75.0%)	0.1667 (16.7%)	0.5833
0.85	1	0	3	12	0.2500 (25.0%)	0.0000 (0.0%)	0.2500
0.95	0	0	4	12	0.0000 (0.0%)	0.0000 (0.0%)	0.0000

Conclusion & Decision Rationale:

- At $T = 0.95$, the system suffers from **complete operational failure** (TPR = 0%), failing to detect a single requirement match.
- At $T = 0.85$, the threshold is excessively conservative: while FPR = 0%, it misses 75% of valid matches (TPR = 25%), which is unacceptable in testing and traceability verification.
- At $T = 0.75$, the system correctly identifies 75% of all true matches (TPR = 0.75) while maintaining a very low false positive rate (FPR = 16.7%). In ROC space, it maximizes **Youden's J-statistic** ($J = 0.5833$). Thus, $T = 0.75$ is **unequivocally the most dependable threshold**.

Solution to Question 11: Prompting Taxonomy and In-Context Learning

Official Correct Choice: b

“Few-shot prompting”

1. In-Context Learning Taxonomy (Brown et al., NeurIPS 2020)

In-Context Learning (ICL) refers to the capability of pre-trained language models to condition their generation on conditioning demonstrations supplied within the context window without updating gradient weights. Prompting strategies are categorized by the number and structure of exemplars:

Taxonomy of Prompting Techniques

- **Zero-Shot Prompting:** The prompt specifies only the task description and the query input without any demonstration examples:

$$\mathcal{P}_{\text{zero}} = \{\text{Instruction}, x_{\text{test}}\}$$

- **One-Shot Prompting:** The prompt provides exactly $k = 1$ demonstration input-output pair before the query:

$$\mathcal{P}_{\text{one}} = \{(x_1, y_1), x_{\text{test}}\}$$

- **Few-Shot Prompting:** The prompt provides $k \geq 2$ demonstration input-output pairs ($k \in [2, 10]$ typically) that establish task framing, syntax, and output schema:

$$\mathcal{P}_{\text{few}} = \{(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k), x_{\text{test}}\}$$

- **Chain-of-Thought (CoT) Prompting:** Demonstrations explicitly contain step-by-step intermediate reasoning paths (r_i) preceding the final answer (y_i):

$$\mathcal{P}_{\text{CoT}} = \{(x_i, r_i, y_i)_{i=1}^k, x_{\text{test}}\}$$

2. Analysis of the Prompt in Question 11

The given prompt presents:

- **Example 1:** Input: “The battery lasts all day...” → Output: Positive
- **Example 2:** Input: “The application crashes...” → Output: Negative
- **Example 3:** Input: “The interface is acceptable...” → Output: Neutral
- **Target Query:** “The camera quality is fantastic...” → [To be classified]

Because the prompt provides exactly $k = 3$ complete, diverse input-output exemplars covering the entire label space (Positive, Negative, Neutral), this is the textbook definition of **Few-shot prompting**.

3. Analysis of Distractors

- **Option a is INCORRECT:** One-shot prompting requires exactly one example ($k = 1$), whereas 3 examples are provided.
- **Option c is INCORRECT:** “Priming” is a broad psychological term describing passive exposure to stimulus words; it is not the formal technical category for structured k -exemplar demonstrations in LLM literature.
- **Option d is INCORRECT:** Zero-shot provides zero examples ($k = 0$).
- **Option e is INCORRECT:** Chain-of-Thought requires intermediate reasoning steps/rationales ($x \rightarrow \text{Rationale} \rightarrow y$). In this prompt, each input directly maps to a discrete sentiment label with zero reasoning tokens.